

 Almanax
Security Scan Report

daccred/attest.so@287e2ed581c99cfe781796344d6aac026c

Status: COMPLETE
Report Generated: 01/12/2025, 11:04:25
Scan Date: Dec 1, 2025 at 10:50 AM

Findings Summary

Severity	Count
● Critical	0
● High	1
● Medium	5
● Low	5
● Info	0
Total	11

contracts/stellar/resolvers/tests/token_reward.rs

● MEDIUM Unchecked addition for reward totals

Description:

In `onresolve`, the contract updates cumulative and per-user reward totals using unchecked `total + reward_amount` and `user_total + reward_amount`. If these sums exceed `i128::MAX`, this will overflow and trap, potentially reverting execution and preventing further reward distributions.

Affected Code:

```
// STEP 5: Update total rewards distributed
let total: i128 = env.storage().instance().get(&DataKey::TotalRewarded).unwrap_or(0);
env.storage().instance().set(&DataKey::TotalRewarded, &(total + reward_amount));

// STEP 6: Update individual user reward totals
let user_key = (DataKey::UserRewards, at
... (truncated)
```

Recommendation:

Use checked arithmetic (e.g., `total.checked_add(reward_amount)`) and return a clear error (such as `ResolverError::ValidationFailed`) on overflow to prevent traps and ensure graceful failure.

● LOW Negative fund amount not validated

Description:

The `fund_reward_pool` function does not validate that the `amount` parameter is non-negative. A malicious or compromised admin could pass a negative value, potentially causing unintended behavior in token transfers (e.g., underflow or reversed transfers), leading to pool corruption or unauthorized withdrawals.

Affected Code:

```
pub fn fund_reward_pool(env: Env, admin: Address, amount: i128) -> Result<(), ResolverError> {
    Self::require_admin(&env, &admin)?;

    // Get reward token address
    let reward_token: Address = env
        .storage()
        .instance()
        .get(&DataKey::RewardToken)
        .ok_or(ResolverError::ValidationFailed)
        ... (truncated)
```

Recommendation:

Add validation to ensure `amount >= 0`, returning an error (e.g., `ResolverError::ValidationFailed`) if the value is negative, similar to checks in `initialize` and `set_reward_amount`.

● LOW Processed flag not set when rewards are zero

Description:

When `reward\_amount` is set to `0`, `onresolve` returns early without marking the attestation as processed (ProcessedAttestation). This allows the same attestation UID to be processed later (e.g., after the reward amount is increased), leading to retroactive reward distribution.

Affected Code:

```
// EARLY EXIT: No rewards configured (admin set amount to 0)
if reward_amount == 0 {
    return Ok(); // Silent success - no rewards to distribute
}
```

Recommendation:

Always mark the attestation as processed before any early return, including when the reward amount is zero, to ensure idempotent behavior and prevent retroactive claims.

contracts/stellar/protocol/src/instructions/schema.rs

● MEDIUM Unbounded storage of schema definitions

Description:

The `register_schema` function allows any caller to store an arbitrary-length `schema_definition` string (up to the Soroban SDK limit ~64 KiB) in persistent contract storage for each new schema. There is no cap on the number of schemas or limits on the size of each definition. An attacker could repeatedly register unique schemas with near-maximum-size definitions, causing state bloat and denial-of-service via resource exhaustion.

Affected Code:

```
77: pub fn register_schema(  
78:     env: &Env,  
79:     caller: Address,  
80:     schema_definition: String,  
81:     resolver: Option<Address>,  
82:     revocable: bool,  
83: ) -> Result<BytesN<32>, Error> {  
84:     // Require authorization from the caller  
85:     caller.require_auth();  
... 98:     let  
... (truncated)
```

Recommendation:

Enforce a maximum length for `schema_definition` (e.g., 1–2 KiB) and/or a maximum number of schemas per authority. Reject or truncate definitions exceeding the limit before storage.

● LOW No validation of schema definition format

Description:

The `register_schema` instruction accepts any `String` as the schema definition without validating that it conforms to the expected format (e.g., valid JSON or XDR). Downstream components relying on a well-formed schema may panic or misbehave when encountering invalid definitions.

Affected Code:

```
79:     schema_definition: String,  
...  
87:     let schema_uid = utils::generate_schema_uid(env, &schema_definition, &caller, &  
resolver);
```

Recommendation:

Parse and validate `schema_definition` against the expected schema format (e.g., use a JSON schema validator or XDR parser). Return `Error::InvalidSchemaDefinition` on failure.

contracts/stellar/deploy.sh

● MEDIUM Sensitive credentials logged to console

Description:

The script logs sensitive network credentials—specifically the RPC URL (which may include an API key) and the network passphrase—to the console in clear text. These secrets can appear in CI logs or terminal history, leading to unintended exposure.

Affected Code:

```
# In verify_network_configuration (around lines 109-110):
echo "  RPC URL: ${rpc}"
echo "  Passphrase: ${passphrase}"

# In confirmation summary (around lines 571-572):
echo "RPC URL: ${rpc_url}"
echo "Network Passphrase: ${network_passphrase}"
```

Recommendation:

Avoid printing secrets to standard output. Redact or omit sensitive values when logging. For example, replace `echo " Passphrase: ${passphrase}"` and the confirmation block logging `'RPC URL'` and `'Network Passphrase'` with masked output or remove these lines.

● LOW Predictable temporary JSON file allows symlink attack

Description:

The `update_contracts_json` function uses a fixed temporary filename (`'deployments.json.tmp'`) when writing updates. An attacker with write access to the working directory could create a symlink named `'deployments.json.tmp'` pointing to an arbitrary file, causing the script to overwrite sensitive files when `'mv -f'` is invoked.

Affected Code:

```
update_contracts_json() {
    local tmp_json_file="${CONTRACTS_JSON_FILE}.tmp"
    ...
    jq ... > "$tmp_json_file"
    mv -f "$tmp_json_file" "$CONTRACTS_JSON_FILE"
}
```

Recommendation:

Use secure temporary files (e.g., via `'mktemp'`) for JSON updates, or ensure the temporary filename is unpredictable. Validate that `'tmp_json_file'` is not a symlink before writing or use `'mv --no-target-directory'`.

contracts/stellar/resolvers/src/fee_collection.rs

MEDIUM Unrestricted contract initialization allows takeover

Description:

The `initialize` function (lines 35–44) is publicly callable by any address. It only requires the provided `admin` parameter to sign, but does not restrict who can perform the first initialization. An attacker can front-run the intended deployer, call `initialize` with their own address as `admin`, and gain full administrative control over fees, fee token configuration, and the fee recipient.

Affected Code:

```
pub fn initialize(
    env: Env,
    admin: Address,
    fee_token: Address,
    attestation_fee: i128,
    fee_recipient: Address,
) -> Result<(), ResolverError> {
    if env.storage().instance().has(&DataKey::Initialized) {
        return Err(ResolverError::CustomError); // Already initialized
    }

    ... (truncated)
```

Recommendation:

Restrict the `initialize` entrypoint to a predetermined deployer or use a built-in mechanism (e.g., check `env.invoker()` against a hardcoded owner or the deployer) so that only the intended party can perform the initial setup.

● LOW Unchecked i128 additions may overflow balances

Description:

The contract updates both per-recipient and total collected fee counters by performing `collected + attestation\_fee` and `total + attestation\_fee` using raw `i128` addition. Without overflow checks, these operations can wrap or panic in certain build modes if values approach `i128::MAX`, corrupting the accounting.

Affected Code:

```
// Track collected fees for recipient
let collected: i128 = env.storage().persistent().get(&key).unwrap_or(0);
env.storage().persistent().set(&key, &(collected + attestation_fee));
// Update total collected
let total: i128 = env.storage().instance().get(&DataKey::TotalCollected).unwrap_or(0);
env.st
... (truncated)
```

Recommendation:

Use checked arithmetic methods (e.g., `checked\_add`) or validate that sums stay within safe bounds before writing back to storage. Alternatively, use saturating arithmetic to cap values and prevent wrapping.

contracts/stellar/protocol/src/instructions/attestation.rs

● MEDIUM Incorrect `revocable` flag in resolver attestation

Description:

In `create\_resolver\_attestation` (lines 79-82), the `revocable` field is hardcoded to `true` instead of reflecting the actual `schema.revocable` value. This causes the resolver to always see attestations as revocable, which may lead the resolver to incorrectly approve or reject revocation requests based on wrong assumptions.

Affected Code:

```
revocable: true, // Will be set based on schema
```

Recommendation:

Populate the `revocable` field of `ResolverAttestation` using the `schema.revocable` value passed into the function. Pass `schema.revocable` as a parameter to `create\_resolver\_attestation`.

contracts/stellar/protocol/src/instructions/delegation.rs

● HIGH Enable incorrect revocability by mismatched schema check

Description:

In `revoke_by_delegation` (lines 127-132), the code fetches the schema to check its revocability using `request.schema_uid`, but never verifies that this `schema_uid` matches the schema of the attestation being revoked. An attacker could supply an `attestation_uid` for a non-revocable schema but pass a different, revocable `schema_uid` in the request, bypassing the revocability guard and revoking an attestation that should not be revocable.

Affected Code:

```
// Verify schema is revocable
let schema = utils::get_schema(env, &request.schema_uid).ok_or(Error::SchemaNotFound)?;
if !schema.revocable {
    return Err(Error::AttestationNotRevocable);
}
```

Recommendation:

Ensure that the `schema_uid` in the revocation request matches the `schema_uid` stored in the attestation. For example, after fetching the attestation, verify `attestation.schema_uid == request.schema_uid` before checking `schema.revocable`.